Master's Thesis

1 | Introduction

2 | Methods

In this research, we demonstrated the effects of common class imbalance correction methodologies, when used in the context of clinical prediction model development. We focused on clinical prediction models designed for dichotomous risk prediction. A simulation study was implemented to illustrate the effects of imbalance correction methodologies across 18 unique data-generating scenarios (section 2.2). For each scenario, we compared the out-of-sample predictive performance (i.e., model performance on data not used to train the model) of prediction models developed using a two-step procedure (section 2.3).

2.1 | Data Generating Mechanism

In this simulation study, we focused on prediction models for dichotomous risk prediction. Consequently, we generated data comprised of two classes. We refer to the negative class (non-events) and positive class (events) as class 0 and class 1, respectively. Data for each class were generated independently using distinct multivariate normal (mvn) distributions. As shown in equations 1 and 2, we specified a distinct mean and covariance structure for each class. The mean structures for the classes are represented as μ_0 and μ_1 for class 0 and 1, respectively. The covariance matrices are represented as Σ_0 and Σ_1 for class 0 and 1, respectively.

$$\text{Class } \ 0 : \mathbf{X} \sim mvn(\boldsymbol{\mu_0}, \boldsymbol{\Sigma_0}) = mvn(\mathbf{0}, \boldsymbol{\Sigma_0}) \tag{1}$$

$$\text{Class } \ 1 : \mathbf{X} \sim mvn(\boldsymbol{\mu_1}, \boldsymbol{\Sigma_1}) = mvn(\boldsymbol{\Delta_\mu}, \boldsymbol{\Sigma_0} - \boldsymbol{\Delta_\Sigma}) \tag{2}$$

As shown in equation 2, the differences in parameter values between the two classes are represented by $\boldsymbol{\Delta_\mu}$ and $\boldsymbol{\Delta_\Sigma}$; a vector and matrix comprised of the differences in predictor means, and variances/ covariances, between the classes, respectively. We specified no variation in means among predictors within a class, making all elements in $\boldsymbol{\Delta_\mu}$ equivalent; we denote these equivalent elements as δ_μ. Similarly, we specified no variation in predictor variances within a class, making all diagonal elements in the matrix $\boldsymbol{\Delta_\Sigma}$ equivalent; diagonal elements are denoted by δ_Σ.

For class 0, all predictor means were fixed to zero and all variances to 1. For class 1, all means were non-zero and are represented by the vector $\boldsymbol{\Delta_\mu}$ (since class 0 means are all zero). Finally, in both classes, we allowed 80% of the predictors to covary. All non-zero correlations among predictors in each class were set to 0.2. To ensure the correlation among predictors was not stronger in one class, we fixed the correlation matrices of the two classes to be equal. This was accomplished by computing the off-diagonal elements of $\boldsymbol{\Delta_\Sigma}$ such that the correlation matrices of the two classes were equivalent (as shown below)

For instance, with 8 predictors:

the mean and covariance structure for class 0 is,

$$\boldsymbol{\mu_0} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \boldsymbol{\Sigma_0} = \begin{bmatrix} 1 & 0.2 & 0.2 & 0.2 & 0.2 & 0.2 & 0 & 0 \\ 0.2 & 1 & 0.2 & 0.2 & 0.2 & 0.2 & 0 & 0 \\ 0.2 & 0.2 & 1 & 0.2 & 0.2 & 0.2 & 0 & 0 \\ 0.2 & 0.2 & 0.2 & 1 & 0.2 & 0.2 & 0 & 0 \\ 0.2 & 0.2 & 0.2 & 0.2 & 1 & 0.2 & 0 & 0 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

and mean and covariance structure for class 1 is,

$$\boldsymbol{\mu_1} = \begin{bmatrix} \delta_\mu \\ \delta_\mu \\ \delta_\mu \\ \delta_\mu \\ \delta_\mu \\ \delta_\mu \\ \delta_\mu \\ \delta_\mu \end{bmatrix}, \boldsymbol{\Sigma_1} = \begin{bmatrix} 1-\delta_\Sigma & z & z & z & z & z & 0 & 0 \\ z & 1-\delta_\Sigma & z & z & z & z & 0 & 0 \\ z & z & 1-\delta_\Sigma & z & z & z & 0 & 0 \\ z & z & z & 1-\delta_\Sigma & z & z & 0 & 0 \\ z & z & z & z & 1-\delta_\Sigma & z & 0 & 0 \\ z & z & z & z & z & 1-\delta_\Sigma & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1-\delta_\Sigma & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1-\delta_\Sigma \end{bmatrix}.$$

Here, $z = (1 - \delta_\Sigma) * 0.2$, to ensure equivalent correlation matrices between the two classes. Note, the covariance matrices were *not* equivalent between the classes.

In this simulation study, 18 unique data-generating scenarios were selected (see Section 2.2). Parameter values for the data generating distributions (δ_μ and δ_Σ) of each scenario were selected to generate a concordance statistic (C) of 0.85. Under the assumption of normality for all predictors (in each class), the concordance statistic of the data can be expressed as a function of $\boldsymbol{\Delta_\mu}$, $\boldsymbol{\Sigma_0}$ and $\boldsymbol{\Sigma_1}$ [1]. Optimal values of δ_μ and δ_Σ for each scenario were computed analytically, based on the following formula [1]:

$$C = \Phi\left(\sqrt{\boldsymbol{\Delta_\mu}' \, (\boldsymbol{\Sigma_0} + \boldsymbol{\Sigma_1})^{-1} \, \boldsymbol{\Delta_\mu}}\right). \tag{3}$$

In equation (3), Φ represents the cumulative density function of the standard normal distribution; $\boldsymbol{\Delta_\mu}$, $\boldsymbol{\Sigma_0}$ and $\boldsymbol{\Sigma_1}$ maintain their previous definitions. To ensure a unique solution, δ_Σ was fixed at 0.3 for each scenario, while equation (3) was solved to yield the appropriate value of δ_μ in each scenario. Full analytical solution is provided in Appendix A.

Finally, given that data for each class were generated independently, we had direct control over how many observations were generated under each class. The number of observations from the positive class (n_1) was sampled from the binomial distribution with probability equal to the specified event fraction. The number of observations in the negative class (n_0) was then computed as $N - n_1$, where N is the specified sample size.

Parameter values for the data generating distributions of the simulation scenarios are presented in Table 1.

2.2 | Data Generating Scenarios

In our simulation study, 18 (3 x 2 x 2) unique data-generating scenarios were implemented (Table 1). This was achieved by varying the following three characteristics of the data: the event fraction (proportion of patients with an event), the number of predictors and sample size. Event fraction (EF) varied through the set $\{0.5,\ 0.2,\ 0.02\}$ and number of predictors through the set $\{8,\ 16\}$. In all scenarios, data were generated to yield an expected concordance statistic (C) of 0.85. Given the number of predictors, the event fraction and expected concordance statistic, we computed the minimum required sample size for a prediction model developed under these conditions. Sample size calculations were carried out using the R package pmsampsize[2]. The sample size of data used to train the prediction models was then varied through the set $\{\frac{1}{2}N,\ N,\ 2N\}$ implying half the required sample size, adequate samples size and double the required sample size, respectively.

Scenario	No. Predictors	Sample Size	Event Fraction	n_{train}	$n_{validation}$	δ_μ	C
1	8	0.5N	0.50	193	1930	0.60	0.85
2	8	0.5N	0.20	124	1240	0.60	0.85
3	8	0.5N	0.02	899	8990	0.60	0.85
4	8	N	0.50	385	3850	0.60	0.85
5	8	N	0.20	247	2470	0.60	0.85
6	8	N	0.02	1797	17970	0.60	0.85
7	8	2N	0.50	770	7700	0.60	0.85
8	8	2N	0.20	494	4940	0.60	0.85
9	8	2N	0.02	3594	35940	0.60	0.85
10	16	0.5N	0.50	193	1930	0.49	0.85
11	16	0.5N	0.20	247	2470	0.49	0.85
12	16	0.5N	0.02	1797	17970	0.49	0.85
13	16	N	0.50	385	3850	0.49	0.85
14	16	N	0.20	493	4930	0.49	0.85
15	16	N	0.02	3593	35930	0.49	0.85
16	16	2N	0.50	770	7700	0.49	0.85
17	16	2N	0.20	986	9860	0.49	0.85
18	16	2N	0.02	7186	71860	0.49	0.85

Table 1: Summary of simulation scenarios.

2.3 | Model Development

All prediction model in our research were developed according to this two-step procedure. First, data are pre-processed using an imbalance correction technique. Then, the resulting pseudo-balanced data are used to train a classification algorithm.

We implemented a 5 x 6 factorial design, to compare the out-of-sample predictive performance of prediction models developed with 5 imbalance corrections (1 control and 4 imbalance corrections) and 6 classification algorithms. In total, we compare the performance of 30 prediction models, each comprised of a unique combination of an imbalance correction and classification algorithm.

2.3.1 | Imbalance Corrections

The imbalance corrections studied in our simulation included: control, random under sampling (RUS), random over sampling (ROS), synthetic majority over sampling (SMOTE) and synthetic majority over sampling with edited neared neighbors (SENN). **more justification for inclusion** . As determined by a recent systematic review, the most common class imbalance correction currently implemented when developing clinical prediction models is SMOTE[3]. All imbalance corrections and the R packages used for their implementation are summarized in Table 2.

In the control "correction", data were not corrected for imbalance, and moved to second step of model development untouched (the imbalanced data were used to train prediction models). All imbalance correction methodologies were implemented to achieve artificially class balanced data ($EF_{\text{corrected}} \approx 0.5$). RUS achieves class balanced data by randomly disregarding observations from the majority class such that the resulting data are class balanced. ROS generates artificial observations for the minority class by re-sampling observations from the minority class with replacement until a balance is achieved. SMOTE generates artificial observations from the minority class by —-[?] ; in our implementation we specify the number of nearest neighbors for smote (k) to be 5, the default in the package used for implementation[4]. SMOTE - ENN — works like this[?] . We specify the number of nearest neighbors in the SMOTE step (k1) to be 5, and the number of nearest neighbors in ENN steep (k2) to be 3, the software defaults[4].

2.3.2 | Classification Algorithms

Classification algorithms were selected based on a systematic review identifying common algorithms used to develop prediction models in a medical context[3]. These algorithms include: logistic regression, support vector machine, random forest and XGBoost. Additionally, based on literature summarizing common strategies to handle class imbalance[5–8], we included two ensemble learning algorithms designed specifically to handle class imbalance: RUSBoost and EasyEnsemble. The hyperparameters selected for these classification algorithms, and the R packages used for algorithm implementation are summarized in Table 2.

Paragraph about implementation. * lr - no hyperparameter * svm - caret, svmRadial, caret default tune grid (C), tunelength = 3 (default) * rf - caret,custom tune grid - mtry (1:npred), min.node.size (min.node.size = 1:10), splitrule = "gini" * xg - caret default tunegrid, tunelength = 3 (default) * rusb - weak classifier = svm with radial kernel * ee - nothing to tune here

	Method	Abbreviation	Hyperparameters	R Package
Imbalance Corrections:	Random Undesampling	RUS		ROSE[9]
	Random Oversampling	ROS		ROSE[9]
	Synthetic Majority Over Sampling	SMOTE	$k = 5$	IRIC[4]
	SMOTE - Edited Nearest Neighbours	SENN	$k1 = 5$, $k2 = 3$	IRIC[4]
Classification Algorithms:	Logistic Regression	LR		base R[10]
	Support Vector Machine	SVM	kernel = radial, tune grid = A	caret[11]
	Random Forest	RF	tune grid = B	caret[11]
	XGBoost	XG	tune grid = C	caret[11]
	RUSBoost	RB	classifier = svm (radial)	ebmc[12]
	EasyEnsemmble	EE		IRIC[4]

Table 2: Summary of imbalance corrections and classification algorithms implemented.

2.4 | Simulation Methods

Under each simulation scenario, 2000 data sets were generated. Each data set was comprised of training and validation data. The training and validation data were generated independently using identical data generating mechanisms. Validation data sets were generated to be ten times larger than the training data sets. The sample sizes of the training and validation data for each simulation scenario can be found in Table 1.

For each generated data set, 30 (5 x 6) prediction models were developed (as described in section 2.3). All prediction models were trained using the training data. Out-of-sample performance was then assessed using the validation data.

This simulation study was conducted using High Performance Computing – ** insert tech specs**.

2.4 | Performance Metrics

Out-of-sample model performance was assessed using measures of calibration, discrimination and overall performance. All performance metrics were computed using validation data.

Calibration was measured using visual and empirical metrics. We assessed calibration visually by means of flexible calibration plots. For each simulation iteration, a flexible calibration curve was generated with loess regression using ggplot2, for all prediction models[13]. Additionally, calibration intercept and slope were calculated according to their respective definitions in Steyerberg et al. (2010)[14].

The concordance statistic was used to measure model discrimination; computed using the R package pROC[15]. For dichotomous risk prediction, it is equivalent to the area under the Receiver Operator Characteristic curve[14].

Overall performance was measured by scaled Brier score. This metric reflects both model discrimination and calibration and was calculated according its definition in Steyerberg et al. (2010)[14].

For empirical measures of model performance (concordance statistic, scaled Brier score, calibration intercept and calibration slope), the median over the simulation iterations and corresponding Monte Carlo error were reported.

2.5 | Error Handling

Under scenarios where training data were class balanced ($EF = 0.5$), imbalance correction methodologies may not preform reliably. For instance, if the classes are exactly balanced there is nothing to over or under sample, and the algorithm returns an error. In the event of an error with imbalance corrections, the uncorrected data were allowed to proceed to train the classification algorithms. Conversely, when data were extremely imbalanced ($EF = 0.02$) and sample size is small ($\frac{1}{2}N$ some classification algorithms may return an error. In this case, predicted probabilties were stored as "NA".

In our simulation, we save all warnings and errors, and report the frequency with which errors occur for each imbalance correction and algorithm in all simulation scenario. We used the tryCatch.W.E function from simsalapar to save errors and warnings in our simulation[16].

- need to talk about recalibration somewhere *

3 | Results

4 | Disucssion

References

[1] Olga V. Demler, Michael J. Pencina, and Ralph B. D'Agostino Sr. Equivalence of improvement in area under roc curve and linear discriminant analysis coefficient under assumption of normality. *Statistics in Medicine*, 30(12):1410–1418, 2011.

[2] Joie Ensor, Emma C. Martin, and Richard D. Riley. *pmsampsize: Calculates the Minimum Sample Size Required for Developing a Multivariable Prediction Model*, 2022. R package version 1.1.2.

[3] Constanza Navarro, Johanna Damen, Maarten van Smeden, Toshihiko Takada, Steven Nijman, Paula Dhiman, Jie Ma, Gary Collins, Ram Bajpai, Richard Riley, Karel Moons, and Lotty Hooft. Systematic review identifies the design and methodological conduct of studies on machine learning-based prediction models. *Journal of Clinical Epidemiology*, 11 2022.

[4] Bing Zhu, Zihan Gao, Junkai Zhao, and Seppe K.L.M. vanden Broucke. Iric: An r library for binary imbalanced classification. *SoftwareX*, 10:100341, 2019.

[5] Lian Yu and Nengfeng Zhou. Survey of imbalanced data methodologies, 2021.

[6] Satyam Maheshwari, R.C. Jain, and R.S. Jadon. An insight into rare class problem: Analysis and potential solutions. *Journal of Computer Science*, 14(6):777–792, May 2018.

[7] Victoria López, Alberto Fernández, Jose G. Moreno-Torres, and Francisco Herrera. Analysis of preprocessing vs. cost-sensitive learning for imbalanced classification. open problems on intrinsic data characteristics. *Expert Systems with Applications*, 39(7):6585–6608, 2012.

[8] Prabhjot Kaur and Anjana Gosain. Empirical assessment of ensemble based approaches to classify imbalanced data in binary classification. *International Journal of Advanced Computer Science and Applications*, 2019.

[9] Nicola Lunardon, Giovanna Menardi, and Nicola Torelli. ROSE: a Package for Binary Imbalanced Learning. *R Journal*, 6(1):82–92, 2014.

[10] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2021.

[11] Max Kuhn. *caret: Classification and Regression Training*, 2021. R package version 6.0-90.

[12] Hsiang Hao and Chen. *ebmc: Ensemble-Based Methods for Class Imbalance Problem*, 2022. R package version 1.0.1.

[13] Hadley Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016.

[14] Ewout W Steyerberg, Andrew J Vickers, Nancy R Cook, Thomas Gerds, Mithat Gonen, Nancy Obuchowski, Michael J Pencina, and Michael W Kattan. Assessing the performance of prediction models: a framework for traditional and novel measures. *Epidemiology (Cambridge, Mass.)*, 21(1):128–138, 01 2010.

[15] Xavier Robin, Natacha Turck, Alexandre Hainard, Natalia Tiberti, Frédérique Lisacek, Jean-Charles Sanchez, and Markus Müller. proc: an open-source package for r and s+ to analyze and compare roc curves. *BMC Bioinformatics*, 12:77, 2011.

[16] Marius Hofert and Martin Mächler. Parallel and other simulations in R made easy: An end-to-end study. *Journal of Statistical Software*, 69(4):1–44, 2016.